

Running PXDesign — Operations Guide

Target setup, hotspot selection, run modes, interpreting results, campaign management
Written against the IMILGPU2 install (RTX 5070 Ti, 16 GB). Companion to *PXDesign on Blackwell*.

1. Starting a session

```
source /data/imil/envs/pxdesign-venv/bin/activate
cd /data/imil/TEST_SPACE/BINDER_DESIGN/software/PXDesign

export PROTENIX_DATA_ROOT_DIR=$PWD/release_data/ccd_cache
export XLA_PYTHON_CLIENT_PREALLOCATE=false
export XLA_PYTHON_CLIENT_ALLOCATOR=platform

which python      # must be .../pxdesign-venv/bin/python
```

Activation is **not** optional. PXDesign runs ProteinMPNN and AF2 as subprocesses that inherit `PATH`. Invoking the binary by absolute path without activating fails mid-pipeline with `No module named 'colabdesign'`.

2. The five subcommands

Command	GPU	What it does
check-input	No	Validates YAML syntax and content. Instant.
parse-target	No	Parses and crops the target; writes a <code>.cif</code> and a PyMOL <code>.pml</code> for visual inspection.
prepare-msa	No	Populates MSA paths in the YAML. Required for extended mode only.
infer	Yes	Raw backbone diffusion, no sequence design, no scoring.
pipeline	Yes	Full loop: diffusion → ProteinMPNN → AF2-IG (→ Protenix in extended mode) → <code>summary.csv</code> .

3. Writing the target YAML

```
target:
  file: /data/imil/TEST_SPACE/BINDER_DESIGN/targets/1TNF/1TNF.cif
  chains:
    A:
      hotspots: [87, 90, 91]
    B:
      hotspots: [32, 33, 145]
    C: "all"

binder_length: 90
```

Chain semantics

Declaration	Meaning
chain with <code>hotspots:</code>	Targeted. Diffusion is conditioned to place the binder against these residues.
chain as <code>"all"</code> , or listed with no hotspots	Present but not targeted. Occupies space and shapes the surface; the binder cannot sit where it is. This is how you keep a trimer intact while designing against one groove.
<code>crop:</code> <code>[...]</code>	Keeps only the listed residues. Does not renumber or rebuild geometry.
omit <code>crop</code>	Full chain retained.

Cropping is usually the wrong move for multimeric targets. On TNF- α the receptor site is the groove *between* two protomers. Cropping deletes the β -sandwich scaffold that holds those loops in place, and cropping to one chain destroys the groove entirely — the model then designs against a flat artificial face. This was the direct cause of poor designs in early web-portal runs. Give whole chains and let hotspots do the steering.

Residue numbering — verify before every new target

PXDesign reads `label_seq_id` from a `.cif`. BindCraft, PyMOL and most literature use PDB-style `auth_seq_id`. These agree for 1TNF but frequently do not elsewhere, and a silent offset means designing against residues you never chose.

```
python - <<'PY'
import biotite.structure.io.pdbx as pdbx
cat = pdbx.CIFFile.read("/path/to/target.cif").block["atom_site"]
ch, lab, auth = (cat[k].as_array() for k in ("auth_asym_id", "label_seq_id", "auth_seq_id"))
offs = {}
for c, l, a in zip(ch, lab, auth):
    if l in (".", "?"): continue
    offs.setdefault(c, set()).add(int(a)-int(l))
print("auth - label offset per chain:", offs)
PY
```

`{0}` for every chain → use your numbers as-is. Anything else → subtract the offset. Then confirm the residue *identities*:

```
python - <<'PY'
import biotite.structure.io.pdbx as pdbx
cat = pdbx.CIFFile.read("/path/to/target.cif").block["atom_site"]
ch, lab, auth, comp = (cat[k].as_array() for k in
    ("auth_asym_id", "label_seq_id", "auth_seq_id", "label_comp_id"))
want = {(("A",87),("A",90),("A",91),("B",32),("B",33),("B",145))} # edit
seen = set()
for c,l,a,r in zip(ch,lab,auth,comp):
    if l in (".","?"): continue
    if (c,int(l)) in want and (c,l) not in seen:
        seen.add((c,l)); print(f"label {c}{l:>4} = {r:>3}    (auth {c}{a})")
PY
```

4. Choosing hotspots

Hotspots must form one coherent, reachable patch. PXDesign conditions the diffusion directly on them — unlike BindCraft, which treats them as a soft AF2 loss target and tolerates a looser set. A BindCraft hotspot list copied over unchanged can over-constrain PXDesign badly.

Geometric rule of thumb

Max pairwise Cα distance	Verdict
< 25 Å	One coherent site. Reachable by a ~90-residue binder.
25-35 Å	Marginal. Consider splitting or dropping outliers.
> 35 Å	Physically unsatisfiable. Produces junk.

Also drop any hotspot with SASA below ~20 Å² — a buried residue cannot contact anything.

Clustering script

```
python - <<'PY'
import biotite.structure.io.pdbx as pdbx, biotite.structure as struc, numpy as np
from scipy.cluster.hierarchy import linkage, fcluster
from scipy.spatial.distance import squareform

path = "/path/to/target.cif"
arr = pdbx.get_structure(pdbx.CIFFile.read(path), model=1, use_author_fields=True)
arr = arr[struc.filter_amino_acids(arr)]
sasa = struc.sasa(arr, vdwradii="Single")
ca = arr[arr.atom_name == "CA"]

HS = "A73 A75 A77 A87 A90 A91 B21 B23 B32 B33 B65 B67 B113 B115 B145 B146".split()
pts, rsa = {}, {}
for h in HS:
    c, r = h[0], int(h[1:])
    m = (ca.chain_id == c) & (ca.res_id == r)
    pts[h] = ca.coord[m][0]
    res = (arr.chain_id == c) & (arr.res_id == r)
    rsa[h] = float(np.nansum(sasa[res]))

ks = list(pts)
D = np.array([[np.linalg.norm(pts[a]-pts[b]) for b in ks] for a in ks])
for CUT in (22, 26, 30):
    lab = fcluster(linkage(squareform(D), checks=False), "complete"), CUT, "distance")
    print(f"\n== diameter <= {CUT} A ==")
    for c in sorted(set(lab)):
        mem = [ks[i] for i in range(len(ks)) if lab[i] == c]
        if len(mem) < 2: continue
        idx = [ks.index(m) for m in mem]
        print(f"  cluster {c} (n={len(mem)}, diam {D[np.ix_(idx,idx)].max():.1f} A): {' '.join(sorted(mem))}")

print("\nSASA (A^2) - under ~20 is buried:")
for k in sorted(rsa, key=rsa.get):
    print(f"  {k:>6}  {rsa[k]:6.1f}  {'BURIED' if rsa[k] < 20 else ''}")
PY
```

Worked example — 1TNF

Sixteen hotspots carried over from a BindCraft campaign spanned 34.7 Å. Clustering at a 22 Å diameter split them cleanly:

Cluster	Members	Diameter	Interpretation
1	A87 A90 A91 B32 B33 B145 B146	19.4 Å	Canonical TNFR epitope, A/B groove — use this
2	B21 B23	6.7 Å	Isolated loop pair, high SASA (137-157 Å²)
3	A73 A75 A77 B65 B67 B113 B115	17.1 Å	A separate surface — valid as its own campaign

B146 was then dropped for low SASA (16.6 Å²), giving the final set **A: 87, 90, 91 / B: 32, 33, 145**. Those residues are Tyr87, Lys90, Val91, Arg32, Ala33, Ala145 — the 84-91, 29-36 and 143-148 receptor-contact loops.

5. Visual verification

```
pxdesign check-input --yaml $T
pxdesign parse-target --yaml $T -o /tmp/px_debug
# open /tmp/px_debug/*_parsed_target.pml in PyMOL, or the .cif in https://molstar.org/viewer/
```

Cropped regions render blue, hotspots pink sticks. Confirm: one contiguous patch, lining a single groove, facing solvent, nothing pink on untargeted chains. In Mol* read **Sequence ID**, not Auth ID.

System PyMOL on Python 3.12 fails with `No module named 'imp'`. Use a conda PyMOL (`conda run -n pymol-env pymol file.pml`) or Mol*. PXDesign itself never calls PyMOL.

6. Run modes

Mode	Pipeline	MSA?	Measured cost*
infer	Diffusion only	No	~24 s/design
--preset preview	+ ProteinMPNN + AF2-IG complex + AF2 monomer	No	~44 s/design
--preset extended	+ Protenix confidence filter	Yes	est. 70–120 s/design

*Measured on RTX 5070 Ti at N_token = 546 (full TNF trimer + 90-residue binder). Scale roughly with N². PD-L1 at 196 tokens ran ~7.5 s/design in infer.

```
# generation only
pxdesign infer -i $T -o $OUT --N_sample 10

# preview
pxdesign pipeline --preset preview -i $T -o $OUT --N_sample 50 \
  --dtype bf16 --use_fast_ln False --use_deepspeed_evo_attention False

# extended (needs MSA)
pxdesign prepare-msa --yaml $T
pxdesign pipeline --preset extended -i $T -o $OUT --N_sample 50 \
  --dtype bf16 --use_fast_ln False --use_deepspeed_evo_attention False
```

Flags that matter

Flag	Use
--dtype bf16	Halves memory vs fp32. Keep on.
--use_fast_ln False	Verified safe. True requires ninja and JIT-compiles a CUDA kernel — unverified on sm_120.
--use_deepspeed_evo_attention False	Keep False. True needs deepspeed + CUTLASS 3.5.1.
--N_sample	Designs per invocation. Startup is ~26 s, so larger chunks amortise better: 2.6 s/design overhead at N=10, 0.5 s at N=50.

MSAs must be built on the **full-length** target sequence even when cropping, because label_seq_id is defined against the full sequence. Identical chains in a homomer share one MSA — generate once, point every chain at it.

7. Reading summary.csv

Written to <out>/design_outputs/<task>/summary.csv , ranked best-first.

Column	Meaning	Good
AF2-IG-easy-success	Passes BindCraft's thresholds	True
AF2-IG-success	Passes the stricter PXDesign thresholds	True
af2_ip_tm	Interface pTM — confidence in the <i>binding</i>	> 0.5
af2_ipAE	Interface predicted aligned error (Å)	< 10.85
af2_plddt	Complex confidence	> 0.8
af2_monomer_plddt	Binder confidence <i>alone</i> — does it fold unaided?	> 0.8
af2_bound_unbound_RMSD	Conformational change on binding	< 3.5 Å, ideally < 1
af2_complex_pred_design_rmsd	Does AF2 reproduce the diffused pose?	< 3.5 Å
alpha / beta / loop	Secondary-structure fractions	—
Rg	Radius of gyration (Å)	compact

Diagnostic patterns

Signature	Reading
High monomer_plddt, low ip_tm	Binder folds well, interface unconvincing. Normal on hard targets. Generate more, or revisit hotspots/length.
Low monomer_plddt (<0.7), high bound_unbound_RMSD	Binder does not fold on its own. Reject.
Large complex_pred_design_rmsd	AF2 disagrees with the diffused pose. Weak candidate.
beta = 0 across the board	Normal — the model produces helical bundles. Vary binder_length for diversity.

1TNF baseline (N=10, preview, A/B groove, length 90)

Rank	ip_tm	ipAE	plddt	mono_plddt	b/u RMSD	Verdict
1	0.72	11.25	0.87	0.937	0.39	Near-pass — misses ipAE by 0.4
2	0.43	18.86	0.78	0.853	5.25	Weak
3	0.43	19.55	0.85	0.930	0.34	Weak but well-folded
4–10	0.18–0.41	21–27	0.84–0.93	0.74–0.97	0.16–2.61	Fail

Roughly 1 in 10 designs lands above ip_tm 0.4. Expect a handful of genuine passes per thousand on a target of this difficulty.

Chain-permutation output

Lines like {0:0, 1:1, 2:2, 3:3} 1.58 versus {0:1, 1:2, 2:0, 3:3} 20.58 confirm the binder docked at the chain assignment you specified rather than an equivalent site elsewhere on a symmetric target. A low identity-permutation value is what you want.

8. Running a campaign

Batch into independent chunks so a crash costs one chunk, not the run. Vary binder length — the main diversity lever, since output is near-uniformly helical.

```
TGT=/data/imil/TEST_SPACE/BINDER_DESIGN/targets/1TNF
for L in 70 90 110; do
    sed "s/^binder_length:./binder_length: $L/" $TGT/tnf_AB_groove.yaml > $TGT/tnf_AB_len${L}.yaml
done

#!/usr/bin/env bash
set -u
source /data/imil/envs/pxdesign-venv/bin/activate
export XLA_PYTHON_CLIENT_PREALLOCATE=false
export XLA_PYTHON_CLIENT_ALLOCATOR=platform
export PROTENIX_DATA_ROOT_DIR=/data/imil/TEST_SPACE/BINDER_DESIGN/software/PXDesign/release_data/ccd_cache
cd /data/imil/TEST_SPACE/BINDER_DESIGN/software/PXDesign

CAMP=/data/imil/TEST_SPACE/BINDER_DESIGN/campaigns/1TNF_24hrs
TGT=/data/imil/TEST_SPACE/BINDER_DESIGN/targets/1TNF
FASTLN=${FASTLN:-False}
CHUNK=${CHUNK:-50}

mkdir -p "$CAMP/logs"
DEADLINE=$(( $(date +%s) + 24*3600 ))
i=0
while [ "$(date +%s)" -lt "$DEADLINE" ]; do
    for L in 70 90 110; do
        [ "$(date +%s)" -lt "$DEADLINE" ] || break 2
        i=$((i+1)); tag=$(printf '%04d' $i)
        echo "[$(date +%F\ %T)] chunk $tag len=$L"
        pxdesign pipeline --preset preview \
            -i "$TGT/tnf_AB_len${L}.yaml" \
            -o "$CAMP/run_${tag}_len${L}" \
            --N_sample "$CHUNK" --dtype bf16 \
            --use_fast_ln "$FASTLN" --use_deepspeed_evo_attention False \
            > "$CAMP/logs/run_${tag}_len${L}.log" 2>&1 \
            || echo " chunk $tag FAILED -> $CAMP/logs/run_${tag}_len${L}.log"
    done
done
echo "[$(date +%F\ %T)] finished after $i chunks"
```

Monitoring

```
C=/data/imil/TEST_SPACE/BINDER_DESIGN/campaigns/1TNF_24hrs
ls -d $C/run_* | wc -l          # chunks started
find $C -name summary.csv | xargs awk -F, 'FNR>1' | wc -l      # designs scored
find $C -name summary.csv | xargs grep -h ",True," | wc -l     # passing either filter

# best candidates across the whole campaign
find $C -name summary.csv | xargs awk -F, 'FNR>1' | sort -t, -k9 -gr | head -20
```

Throughput planning

Mode	Per design	Per 24 h	Time for 10,000
infer	~24 s	~3,600	~2.8 days
preview	~44 s	~1,950	~5 days
extended (est.)	70-120 s	700-1,200	8-14 days

Upstream guidance is 10,000+ designs to yield 10-100 passing both filters. The efficient shape is **preview broadly, extended on survivors only** — you skip Protenix on the ~95 % that were never going to pass. Disk is a non-issue: ~0.72 MB/design, so 10,000 designs is ~7 GB.

On a 16 GB card the workload is **compute-bound, not memory-bound**: 100 % GPU utilisation at ~240 W with only 6.6 GB of 16 GB used. Batching will not help; the levers are `--use_fast_ln`, larger `--N_sample` chunks, and fewer AF2 recycles.

9. Output layout

```
<out>/
├── pipeline_input.json
├── <TARGET>.pkl.gz          parsed target cache
├── global_run_0/<task>/seed_<N>/
│   ├── predictions/
│   │   ├── <task>_sample_*.cif    diffused backbones
│   │   ├── converted_pdbs/       MPNN inputs
│   │   └── af2_pred/             AF2 complex + monomer models
├── design_outputs/<task>/
│   ├── summary.csv              ranked metrics ← start here
│   ├── orig_designed/rank_*.cif  final structures, best-first
│   ├── server_preview_mode.png  score plots
└── task_info.json
```

10. Troubleshooting at run time

Symptom	Cause	Action
Designs look like spaghetti, or dock nowhere near the intended site	Hotspots too dispersed, or wrong residues from a numbering offset	Run the clustering + numbering checks in §3-4
All <code>iptm</code> < 0.2	Epitope may be undesignable at this binder length	Try 70 / 110; verify hotspots sit in a concave groove, not a flat face
No module named 'colabdesign' mid-run	venv not activated	\$1
CUDA OOM	Target too large for 16 GB	Reduce chains, lower <code>--N_sample</code> , confirm the XLA env vars are set
Ninja is required	<code>--use_fast_ln</code> True	Set False, or <code>uv pip install ninja</code>
<code>summary.csv</code> missing after a run	JSON serialization patch absent	See Patch 3 in the setup guide
<code>libtpu.so</code> / Delay kernel timed out	jax TPU probe / profiler warmup	Harmless

11. Quick reference

```
# session
source /data/imil/envs/pxdesign-venv/bin/activate
cd /data/imil/TEST_SPACE/BINDER_DESIGN/software/PXDesign
export PROTENIX_DATA_ROOT_DIR=$PWD/release_data/ccd_cache
export XLA_PYTHON_CLIENT_PREALLOCATE=false XLA_PYTHON_CLIENT_ALLOCATOR=platform

T=/path/to/target.yaml
O=/data/imil/TEST_SPACE/BINDER_DESIGN/campaigns/myrun

pxdesign check-input --yaml $T
pxdesign parse-target --yaml $T -o /tmp/px_debug
pxdesign infer -i $T -o $O --N_sample 10
pxdesign pipeline --preset preview -i $T -o $O --N_sample 50 \
  --dtype bf16 --use_fast_ln False --use_deepspeed_evo_attention False

find $O -name summary.csv -exec head -5 {} \;
```

Workflow discipline that pays off. (1) Verify numbering and hotspot geometry before any GPU time — both checks are seconds and both failure modes are silent. (2) Run N=10 first on any new target. (3) Snapshot the env with `uv pip freeze` whenever the pipeline completes cleanly. (4) Batch long campaigns into independent chunks.